

MARKETHUB

Multi-Tenant SaaS Marketplace Platform

Laravel + Angular Project Documentation

Project Type: Multi-Vendor / Multi-Tenant E-Commerce SaaS

Backend: Laravel / PHP Frontend: Angular / TypeScript

Database: MySQL

Version 1.0 • September 2026

1. Executive Summary

MarketHub is a multi-tenant SaaS marketplace designed to operate like a digital market where independent sellers can create or purchase access to stores, list different categories of products, manage inventory and orders, receive payments, and interact with customers. The platform owner controls the overall marketplace, subscriptions, commissions, stores, users, payments, and platform settings.

The proposed technology stack uses Laravel as a secure REST API/backend and Angular as the modern web frontend. MySQL will provide relational data storage, while optional Redis, queues, notifications, object storage, and payment integrations can support scale and reliability.

2. Project Vision

The vision is to provide one marketplace where many businesses can independently operate online stores without needing to build and maintain separate e-commerce systems.

Core concept:

One Marketplace → Many Stores → Many Sellers → Many Products → Many Customers

3. User Types

4. Multi-Tenancy Architecture

The first release should use a shared MySQL database with tenant isolation through a `tenant_id` column on tenant-owned records. This is simpler to operate and provides a strong foundation for scaling. Enterprise customers can later be moved to isolated databases if required.

Example structure:

```
marketplace.com
■■■ Store A → Products, Orders, Customers
■■■ Store B → Products, Orders, Customers
■■■ Store C → Products, Orders, Customers
■■■ Store D → Products, Orders, Customers
```

Tenant-owned queries must always be scoped to the authenticated tenant. Authorization policies, middleware, service-layer checks, and database constraints should work together to prevent cross-tenant data access.

5. Store and Tenant Model

- Tenant registration with name, email, phone, password, country, and business details.
- Store creation with store name, unique slug, logo, banner, description, contact details, and address.
- Optional store URLs such as `/store/store-slug`.
- Future support for subdomains such as `store-slug.marketplace.com`.
- Future support for custom domains as a premium feature.
- Store-specific settings, branding, delivery options, and business information.

6. Product Management

The product engine should be category-independent so that a tenant can sell different goods such as electronics, fashion, food, beauty products, automotive parts, books, sports goods, and household products.

- Product name, SKU, description, pricing, status, category, brand, tax information, and stock.
- Multiple product images.
- Product variants such as size, color, capacity, model, or package type.
- Variant-specific SKU, price, stock, barcode, and images where required.
- Categories and subcategories.
- Product search, filtering, sorting, and featured products.
- Low-stock and out-of-stock indicators.

7. Categories and Attributes

Platform administrators can manage marketplace categories and subcategories. Product attributes should be flexible enough to support different industries instead of assuming that every product has the same fields.

Example:

Electronics → Mobile Phones → Smartphones → Product

8. Inventory Management

- Current stock quantity.
- Stock adjustments.
- Inventory movement history.
- Stock-in and stock-out records.
- Low-stock thresholds.
- SKU and barcode support.
- Variant-level inventory.
- Optional warehouse/location support for future releases.

9. Marketplace and Storefront

- Marketplace home page with categories, featured stores, featured products, and search.
- Individual store pages with store banner, logo, description, categories, and products.
- Responsive product listing and product detail pages.
- Product images, variants, pricing, stock, specifications, reviews, and seller information.
- Wishlist/favorites.
- Customer search and filtering.
- Related and recommended products.

10. Shopping Cart and Checkout

The cart should support products from multiple stores. At checkout, the system can create one master customer order and split it into seller-specific orders for fulfillment and seller accounting.

Master Order
 ■■■ Seller Order #1001 → Store A
 ■■■ Seller Order #1002 → Store B
 ■■■ Seller Order #1003 → Store C

11. Orders

12. Payments

The payment architecture should use a gateway abstraction instead of hard-coding one provider. This allows the platform to support Ghana-focused payment methods as well as cards, bank payments, and additional gateways as the business expands.

PaymentService → Gateway A / Gateway B / Gateway C

- Payment transaction records.
- Payment status and reference numbers.
- Webhook handling.
- Idempotency protection for repeated payment notifications.
- Order-payment reconciliation.
- Refund support.
- Seller payout and platform commission calculations.

13. SaaS Subscription System

These limits are illustrative and should be configurable from the Super Admin panel rather than hard-coded.

14. Platform Revenue Model

- Recurring tenant subscriptions.
- Marketplace commission on completed sales.
- Featured products and promoted listings.
- Featured stores.
- Sponsored category or banner advertising.
- Optional premium services such as custom domains, advanced analytics, or higher storage.

Example commission:

Product price = GH₵500 → Marketplace commission = 5% → Platform = GH₵25 → Seller = GH₵475

15. Seller Staff and RBAC

- Store Owner.
- Store Manager.
- Sales Staff.
- Inventory Staff.
- Accountant.
- Custom roles and permissions.
- Permissions for products, orders, customers, inventory, reports, payments, and settings.

16. Delivery and Fulfillment

- Customer pickup.
- Store delivery.
- Third-party courier.
- Platform-managed delivery.
- Delivery zones and fees.
- Delivery assignment and tracking.

- Future driver and vehicle management.

17. Reviews, Ratings and Trust

- Verified customer product reviews.
- Product star ratings.
- Store ratings.
- Review moderation.
- Report/flag review functionality.
- Seller profile and store information to improve customer trust.

18. Super Admin Dashboard

- Dashboard KPIs: tenants, active stores, customers, products, orders, revenue, commission, and payouts.
- Tenant management.
- Store management.
- Customer management.
- Product and category moderation.
- Orders and payment monitoring.
- Subscription plans and billing.
- Commissions and seller payouts.
- Reviews and disputes.
- Coupons and advertisements.
- Reports and analytics.
- Global platform settings.

19. Tenant Dashboard

- Today's sales.
- Orders.
- Products.
- Customers.
- Low-stock products.
- Sales charts.
- Recent orders.
- Top products.
- Recent payments.
- Store performance and analytics.

20. Customer Features

- Registration and login.
- Profile management.
- Multiple addresses.
- Product browsing and search.
- Shopping cart.
- Wishlist.
- Checkout and payments.
- Order history.
- Order tracking.
- Product and store reviews.
- Notifications.

21. Recommended Technology Stack

22. Laravel Backend Modules

```
Authentication
Multi-Tenancy
Users
Roles & Permissions
Stores
Products
Categories
Product Variants
Inventory
Customers
Cart
Orders
Payments
Subscriptions
Commissions
Seller Payouts
Reviews
Coupons
Delivery
Notifications
Reports
Admin
```

23. Suggested Angular Structure

```
src/app/
├── core/
│   ├── auth/
│   ├── guards/
│   ├── interceptors/
│   ├── services/
│   └── shared/
│       ├── components/
│       ├── directives/
│       ├── pipes/
│       ├── features/
│       ├── marketplace/
│       ├── customer/
│       ├── seller/
│       └── admin/
```

24. Initial Database Design

25. Security Requirements

- Strict tenant isolation.
- Role-based authorization and Laravel policies.
- Secure password hashing.
- API authentication and token expiration strategy.
- CSRF protection where applicable.
- Input validation and request sanitization.
- Rate limiting for sensitive APIs.
- Secure file upload validation.
- Payment webhook signature verification.
- Audit logging for sensitive administrative actions.
- Encrypted secrets and environment configuration.

- Protection against SQL injection, XSS, IDOR, and unauthorized resource access.

26. Development Phases

27. Recommended MVP Navigation

Public: Home | Categories | Stores | Products | Search | Login | Register

Seller: Dashboard | Orders | Products | Inventory | Customers | Analytics | Staff | Store Settings | Subscription

Admin: Dashboard | Tenants | Stores | Customers | Products | Categories | Orders | Payments | Plans | Commissions | Payouts | Reviews | Disputes | Reports | Settings

28. Future Enhancements

- Progressive Web App or native mobile application.
- Custom domains and branded storefronts.
- Advanced seller analytics.
- AI-assisted product descriptions and categorization.
- AI-powered recommendations.
- Advertising and sponsored product engine.
- Marketplace API and webhooks.
- Enterprise isolated database option.
- Warehouse and multi-location inventory.
- Advanced delivery logistics.
- Loyalty and rewards.
- Affiliate and referral system.

29. Project Success Criteria

- A tenant can register and create a store without administrator intervention after approval rules are satisfied.
- A tenant can list different types of products and manage variants and stock.
- Customers can discover products across multiple stores.
- Customers can place an order containing products from multiple sellers.
- The platform can calculate commissions and seller earnings accurately.
- Tenant data remains isolated from other tenants.
- Administrators can manage subscriptions, stores, users, transactions, and platform settings.
- The architecture can support additional payment gateways and future mobile clients.

30. Conclusion

MarketHub should be developed as a scalable multi-tenant SaaS rather than as a single-store e-commerce website. Laravel should provide the secure business logic and API layer, while Angular should provide the responsive marketplace, seller dashboard, customer experience, and administrative interfaces. The first development priority should be the database model, tenant isolation, authentication/authorization, product architecture, order lifecycle, and

payment/commission design. Once those foundations are stable, the visual experience and advanced SaaS features can be expanded safely.